Vortrag „Software Testing mit KI“: Gliederung (Entwurf v1)

Rahmen: 60 Minuten, 2 Vortragende, klassischer Aufbau ohne Hook. **Demo-App:** Prüfprot (Altklausuren-Plattform). Die Demos werden vorher als Videos aufgenommen.

Legende: ohne Markierung = eigene Themen, 💡 = Vorschlag zur Ergänzung, ❓ = noch offen

---

Aufbau jedes Themas

Jedes Thema folgt demselben Schema, damit der Vortrag einheitlich wirkt:

1. **Was ist das, und wozu dient es?** Allgemein und ohne KI erklärt
2. **Klassisches Vorgehen:** typische Tools und Methoden
3. **KI-Einsatz:** Demo-Video an Prüfprot
4. **Vor- und Nachteile:** Was kann die KI gut, wo liegen die Grenzen, welche Besonderheiten gibt es?
5. **Skill-Sammlung:** Was wir aus dem Thema als Skill festhalten (siehe 2.2)

Teil 1: Grundlagen (ohne KI)

1.1 Einführung

- Worum geht es heute, und wie läuft der Vortrag ab?
- Was ist Testen, und wofür nutzt man es? (Fehler finden, Vertrauen schaffen, Refactoring absichern, Spezifikation dokumentieren)
- Kosten eines Fehlers je nach Phase (früh gefunden = billig)
- Vorstellung der Demo-App Prüfprot (kurz: Upload, Suche, Voting)

1.2 Testebenen: die Testpyramide

- Unit → Integration → E2E: Kosten, Geschwindigkeit und Aussagekraft je Ebene
- Integration unterteilt in:
  - **Backend-/API-Tests** (Controller + DB, echtes Postgres)
  - **UI-Tests** (Frontend-Komponenten, Zusammenspiel mit der API)
- 💡 **Testebenen vs. Testarten trennen:** Regression ist keine eigene Ebene, sondern ein *Zweck*, der quer über alle Ebenen läuft. Vorschlag für eine Folie:
  - Ebenen: Unit, Integration, E2E
  - Arten: funktional, Regression, Smoke, nicht-funktional (Performance/Last, Security, Usability, Accessibility)

1.3 Testprinzipien und Methoden

- Test-Driven Development (Red → Green → Refactor)
- 💡 BDD / Given-When-Then, Arrange-Act-Assert

- 💡 Black-Box vs. White-Box: Testet man gegen die Spezifikation oder gegen den Code? Diese Frage wird später bei der KI zentral.
- 💡 Äquivalenzklassen und **Grenzwertanalyse** (Beispiel: genau 10 MB Upload, 5 Stimmen fürs Einklappen)
- 💡 Gute Tests: F.I.R.S.T. (Fast, Independent, Repeatable, Self-validating, Timely)
- 💡 Die 7 Testprinzipien nach ISTQB, insbesondere:
  - „Testen zeigt die Anwesenheit von Fehlern, nicht ihre Abwesenheit“
  - **„Vollständiges Testen ist unmöglich“** → hier kommt der **Komplexitätsgraph** (siehe unten)

Komplexitätsgraph: Man kann nicht alles testen

- Graph: Anzahl der Testfälle in Abhängigkeit von der Zahl der Features/Parameter (kombinatorische Explosion)
- Beispiel Prüfprot: 7 Suchfilter mit je n Werten × Sortierung × Votes × Einklapp-Zustand …
- 💡 Platzierung: **in 1.3 einführen und am Ende in Teil 6 noch einmal aufgreifen**. Das gibt dem Vortrag einen Bogen: „Darum braucht man Priorisierung, Risiko-basiertes Testen, Menschen und Monitoring in Produktion.“
- ❓ Mit echten Zahlen aus Prüfprot oder abstrakt (n Parameter mit k Werten → k^n)?

Teil 2: KI im Testing, Grundlagen

2.1 Wie ein KI-Agent testet

- Feedback-Loop: schreiben → ausführen → Fehler lesen → korrigieren
- Kontext: Spezifikation im Repo (CLAUDE.md), Leitplanken
- 💡 **Das Orakel-Problem:** Woher weiß die KI, was „richtig“ ist? Aus der Spezifikation oder aus dem Code? (Verbindung zu Black-/White-Box aus 1.3)

2.2 Skills: Testwissen festhalten

- Was ist ein Skill: Domänen- und Projektwissen einmal aufschreiben statt in jedem Prompt
- Idee: eine **Skill-Sammlung** mit einem Skill pro Testart, die im Lauf des Vortrags wächst
  - z. B. `unit-testing`, `api-testing`, `e2e-playwright`, `ui-visual-check`, `security-testing`, `load-testing`, `test-review`
- 💡 Die Skill-Sammlung könnte der **rote Faden** sein: Jedes Kapitel endet mit „Das kommt in den Skill“.
- 💡 Vergleich in einer Demo: derselbe Auftrag mit und ohne Skill
- ❓ SPARQL-Skill als Beispiel für einen Spezial-Skill: noch relevant?

Teil 3: Durch die Ebenen mit KI

3.1 Unit-Tests

- Allgemein: isoliert, schnell, viele davon
- KI-Demo: Tests aus der Spezifikation (Validierung, Semesterformat, Ranking)
- 💡 Grenzwerte: Findet die KI sie von selbst oder erst auf Hinweis?
- 💡 Property-based Testing (jqwik) fürs Ranking: Eigenschaften statt Beispiele

- 💡 Testdaten-Generierung mit KI
- Vor- und Nachteile

3.2 Integrationstests: Backend/API

- Allgemein: Zusammenspiel mehrerer Komponenten, echte Infrastruktur
- Klassisch: Testcontainers mit echtem Postgres (warum kein H2)
- KI-Demo: Upload → Liste → Download, Statuscodes, Problem-JSON
- 💡 Nebenläufige Votes als Beispiel für Fehler, die nur mit echter DB auffallen
- 💡 Agent testet die API direkt per curl/HTTP mit einem API-Skill
- 💡 optional: Contract Testing Frontend ↔ Backend
- Vor- und Nachteile

3.3 UI-Tests: Funktionalität

- Allgemein: Funktioniert die Oberfläche wie gewünscht?
- Klassisch: Playwright (kurzer Bogen Selenium → Playwright), `data-testid`
- KI-Demo A: Die KI **schreibt** Playwright-Tests (deterministisch und wiederholbar)
- KI-Demo B: Der Agent **bedient** die App selbst im Browser (Browser-MCP), klickt und prüft
- 💡 Exploratives Testen: „Versuch, die App kaputtzumachen“
- 💡 Komponententests (Vitest + Vue Test Utils) als günstigere Ebene unterhalb von E2E
- Vor- und Nachteile: deterministisch vs. flexibel, Kosten, Reproduzierbarkeit

3.4 UI-Tests: Aussehen

- Allgemein: Sieht die Oberfläche richtig aus? (Layout, Responsiveness, Darstellung)
- Klassisch: visuelle Regression per Screenshot-Diff (Pixelvergleich)
- KI-Demo: Agent macht Screenshots und beurteilt das Layout (z. B. Desktop vs. Handy, ausgegraute Einträge)
- 💡 Accessibility-Checks (axe) und KI-Beurteilung von Barrierefreiheit
- Vor- und Nachteile: Pixel-Diff ist streng, aber fehleranfällig; die KI ist tolerant, aber unscharf

3.5 Regressionstests und Bugs verifizieren

- Allgemein: Bereits behobene Fehler dürfen nicht zurückkommen
- KI-Demo: Bug-Meldung → reproduzierender Test (rot) → Fix → Test bleibt als Regressionstest
- 💡 Characterization Tests (Golden Master) vor einem Refactoring

---

Teil 4: Nicht-funktionales Testen mit KI

4.1 Security Testing

- Allgemein: SAST, DAST, Penetrationstests, OWASP Top 10
- KI-Demo: Agent sucht gezielt Schwachstellen (Uploads, Dateinamen, Header, Eingaben)
- 💡 Vergleich: klassische Tools (z. B. Semgrep, CodeQL, OWASP ZAP) vs. KI-Security-Review
- 💡 Fuzzing (z. B. Jazzer) als deterministische Ergänzung
- Limitations: Halluzinierte Findings, falsche Sicherheit, die KI findet nicht alles, rechtliche und ethische Grenzen

4.2 Last- und Stresstests

- Allgemein: Last-, Stress-, Spike- und Soak-Tests; Kennzahlen (Latenz, Durchsatz, Fehlerrate)
- Klassisch: k6 / Gatling / JMeter
- KI-Demo: Agent erzeugt ein k6-Skript aus der API-Beschreibung und wertet die Ergebnisse aus
- Vor- und Nachteile: Skripte schreibt die KI schnell; realistische Lastprofile und die Interpretation brauchen Fachwissen

---

Teil 5: Qualität der KI-Tests und Automatisierung

5.1 Typische Fehler von KI und Agenten

- Tests spiegeln die Implementierung statt der Anforderung
- Die KI „schummelt“: Assertion anpassen, `@Disabled`, Sonderfall im Code einbauen
- Alles wegmocken, Tests ohne echte Aussage
- Halluzinierte APIs/Methoden, falsche Annahmen über das Verhalten
- Falsch-positive Erfolgsmeldung („alle Tests grün“, obwohl sie nicht gelaufen sind)
- 💡 **Gegenmittel:** Leitplanken in der CLAUDE.md, Hooks (Tests laufen automatisch, Testdateien gesperrt)
- 💡 **Mutation Testing (PIT):** Coverage vs. Mutation Score. Würde der Test fehlschlagen, wenn der Code falsch wäre? (ist im Projekt schon eingerichtet)

5.2 Testen mit mehreren Agenten (Subagents)

- Aufteilung der Rollen: Test-Agent, Implementierungs-Agent, Review-Agent
- 💡 **Clean-Room-Testing:** Der Test-Agent sieht nur die Spezifikation, nicht den Code
- 💡 Paralleles Testen: je ein Subagent pro Ebene oder Testart (unit, api, ui, security)
- Vor- und Nachteile: unabhängigere Perspektiven vs. Tokenkosten und Koordination

5.3 Testing und Review in CI/CD

- Allgemein: Pipeline-Stufen (Build → Unit → Integration → E2E → Deploy), Quality Gates
- KI im PR-Review: Review-Agenten, Skills bzw. Masterprompt für Reviews
- 💡 Statische Code-Analyse in der Pipeline (Linter, SpotBugs/Error Prone, Sonar) + KI zum Einordnen der Findings
- 💡 Flaky Tests: Erkennen und Diagnostizieren mit KI
- Diskussion: Darf die KI in der CI Tests selbst „reparieren“?

5.4 (optional) Testmigration: deterministische Tools vs. KI

- JUnit 4 → JUnit 5: OpenRewrite vs. KI
- Vergleich: Determinismus, Tokenkosten, Reproduzierbarkeit, Sonderfälle
- Ergebnis: hybrider Ansatz (erst das Tool, dann die KI für die Reste)
- ❓ Im Hauptteil oder als Backup und für Fragen?

Teil 6: Grenzen und Fazit

6.1 KI ersetzt keine echten Testumgebungen

- Staging- bzw. produktionsnahe Umgebungen
- Menschen testen durch: explorativ, User Acceptance Tests, Usability
- Testen in Produktion: A/B-Tests, Canary Releases, Feature Flags, Monitoring
- **Immer rollback-fähig bleiben**
- Rückgriff auf den **Komplexitätsgraph**: Man kann nicht alles vorher testen, deshalb braucht es diese Sicherheitsnetze

6.2 Fazit und Ausblick

- 💡 Checkliste zum Mitnehmen: Wie prüfe ich KI-geschriebene Tests?
- 💡 Ausblick: Evals für LLM-Features (wenn die App selbst KI nutzt)
- Kernbotschaft: ❓ (z. B. „Die KI beschleunigt das Testen, die Verantwortung für die Qualität bleibt beim Menschen“)